

Horana Urban Council System

Final Project Report



Sri Lanka Institute of Information Technology

IT2080 Information Technology Project

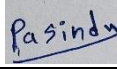
Group ITP_WD_B3_52

October 2025

Declaration

This project report is our original work and the content is not plagiarized from any other resource. References for all the content taken from external resources are correctly cited. To the best of our knowledge, this report does not contain any material published or written by third parties, except as acknowledged in the text.

Authors:

Author SID	Author name	Signature
IT23848184	CHAMYA N D	
IT23817562	S D A S AMARAJEEWA	
IT23815582	KAVISHKA S P P	
IT23711228	SUBASINGHE S M C D	
IT23580008	HEWAGE S H	

Date: 14/10/2025

Abstract

The **Horana Urban Council Smart Management System** was developed to overcome the limitations of the existing manual processes by introducing a centralized, web-based platform for managing citizen services. The system integrates key modules including Complaint Handling, Permit and License Management, Service Booking with conflict prevention, Online Payments, HR and Payroll, and Inventory Management. Built using the MERN stack (MongoDB, Express, ReactJS, and NodeJS), the platform provides a responsive, scalable, and user-friendly interface. Security measures such as role-based access control, JWT authentication, and encrypted data storage were implemented to ensure reliability and data integrity. The project followed the **Agile methodology**, allowing development in iterative sprints with continuous feedback from stakeholders. Comprehensive testing, including unit, integration, and user acceptance testing, validated the system's effectiveness. The HUC-SMS enhances efficiency, transparency, and citizen satisfaction while laying a strong foundation for future improvements such as multilingual support, mobile applications, and blockchain-based governance.

Acknowledgement

We would like to express our sincere gratitude to our supervisor, **Mr. Harshanath**, for his invaluable guidance, encouragement, and continuous support throughout the development of this project. His constructive feedback, expert knowledge, and patience greatly contributed to shaping this system into a practical and impactful solution.

We also extend our appreciation to the lecturers and staff of our department for providing the academic foundation and technical resources that enabled us to successfully complete this work.

Special thanks go to our project team members for their collaboration, dedication, and teamwork during every phase of development, as well as to all stakeholders who shared their insights and requirements, which were essential for designing a system that addresses real-world needs.

Finally, we would like to acknowledge our families and friends for their encouragement, understanding, and support throughout this journey.

Contents

Chapter 1 – Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.2 Motivation	2
1.4 Literature Review	2
1.5 Aim	2
1.6 Objectives.....	3
1.7 Solution Overview	3
1.8 Methodology.....	3
1.9 The Structure of the Report	7
Chapter 2. Requirements.....	8
2.1 Stakeholder analysis	8
2.2 Requirements Analysis	9
2.2.1 Functional Requirements	9
2.2.2 Non-Functional Requirements	10
2.3 Requirements Modeling.....	11
2.3.2 Activity Workflows.....	11
2.3.3 Data Model	12
2.4 Technical Requirements.....	12
2.4.1 Software Requirements.....	12
2.4.2 Network Requirements	13
2.4.3 Security Requirements	13
3.1 System Architecture	14
3.2 Diagrams of Components	16
3.2.1 Kavishka S P P (IT23815582)- Complaint Management Module	16
3.2.2Amarajeewa S D A S (IT23817562) Service Booking Module (Playground & Crematorium).....	16
3.2.3 Chamya N D (IT23848184) Online Payments & Finance Module	22
3.2.4 HR & Payroll Module.....	26
3.2.5 Inventory Management Module	27
3.4 Database Design.....	32

3.5 User Interface Design	33
3.6 Algorithms	33
Chapter 4 – Implementation	34
4.1 Technology Stack	34
4.2 Module Implementations	34
4.2.1 Complaint Management Module	34
4.2.2 Permit & License Module	35
4.2.3 Service Booking Module	36
4.2.4 Online Payments & Finance.....	36
4.2.5 HR & Payroll Module.....	37
4.2.6 Inventory Management Module	37
4.3 API Structure	37
4.4 Authentication & Security.....	38
4.5 Deployment	38
Chapter 5 – Testing	39
5.1 Test Case Design	39
5.1.1. Financial & Payment Module.....	39
5.1.2. Property Assessment & Shop Module	40
5.1.3 Service & License Permit System	40
5.2 Test Results.....	41
5.3 Bug Fixes and Improvements	41
Chapter 6 – Evaluation and Conclusion.....	42
6.1 Evaluation Against Objectives.....	42
6.3 SWOT Analysis.....	42

List of Figures

Figure 1- Onion Diagram.....	8
Figure 2- System Flow Diagram.....	11
Figure 3-Database Schema	12
Figure 4- System Diagram.....	15
Figure 5- Use Case Diagram for Complaint	16
Figure 6- Sequence Diagram for Submit Complaint	17
Figure 7- Activity diagram for Submit Complaint.....	18
Figure 8- Use case Diagram for Service	19
Figure 9- Sequence diagram for Booking sytem	20
Figure 10- Activity diagram for Booking a service	21
Figure 11- Use Case Diagram for Payment Module.....	23
Figure 12- Sequence Diagram for Payment Module	24
Figure 13- Activity diagram for Payment Module.....	25
Figure 14- Activity diagram for Shop Property Application	25
Figure 15- Use Case Diagram for salary.....	26
Figure 16- Sequence Diagram for salary calculation.....	27
Figure 17- Activity Diagram for salary management	28
Figure 18- Use Case Diagram for inventory	29
Figure 19- Sequence Diagram for add inventory item.....	30
Figure 20- Activity Diagram for add inventory item.....	31

List of Tables

Table 1- Test Case for Financial & Payment Module	39
Table 2- Test Case Property Assessment & Shop Module	40
Table 3- Test Case for Service & License Permit System	40
Table 4- Evaluation Against Objectives	42

Chapter 1 – Introduction

1.1 Background

Urban Councils in Sri Lanka play a vital role in providing essential public services to local citizens. These responsibilities include issuing permits and licenses, managing property assessments, collecting taxes and fines, handling citizen complaints, managing public service resources, and paying employee salaries. Despite the importance of these functions, many urban councils still rely heavily on manual, paper-based systems.

The Horana Urban Council (HUC) is one such institution where critical municipal operations are handled through outdated methods. Citizens are required to physically visit the council office to submit complaints, apply for licenses, or make payments. Records are stored in physical files, and communication between citizens and officers is limited to in-person interactions or occasional phone calls.

This outdated approach causes several issues:

- Delays in service delivery - Citizens spend hours in queues, while officers spend excessive time on manual paperwork.
- Errors in records - Paper-based entries often result in mistakes, missing data, or lost documents.
- Limited accessibility - Services are only available during office hours, which is inconvenient for many citizens.
- Lack of transparency - Citizens have no way to check the progress of their applications or complaints.
- Payroll difficulties - Employee leave, overtime, and salaries are tracked manually, causing inefficiencies and disputes.

The limitations of this system highlight the urgent need for a digital transformation of municipal services. By moving from manual to automated systems, HUC can improve efficiency, transparency, and citizen satisfaction.

1.2 Problem Statement

Currently, the Horana Urban Council's reliance on paper-based operations results in:

- Long queues for payments and form submissions.
- No ability for citizens to track requests or complaints.
- Increased risk of errors and document loss.
- Lack of accessibility for citizens outside office hours.

- Inefficient HR management for staff salaries, leave, and overtime.
- These problems negatively affect citizen trust, operational efficiency, and service quality.

1.2 Motivation

The motivation behind this project is to design a Smart Urban Council Management System that digitizes HUC's critical services. Inspired by the success of digital transformation in other sectors, the project aims to provide citizens with easy, transparent, and reliable services while empowering council officers with automated workflows and real-time reporting.

Key motivating factors include:

- Efficiency – Reduce time taken for service delivery.
- Accessibility – Provide 24/7 availability through a web portal.
- Transparency – Enable citizens to track applications and complaints.
- Innovation – Introduce modern features like chatbot booking and multilingual forms.
- Scalability – Create a solution that can be adapted by other urban councils in Sri Lanka.

1.4 Literature Review

A review of existing digital governance systems shows that while councils like Colombo MC and Gampaha MC have introduced limited e-services (mainly payments), they lack comprehensive systems for complaints, HR, licensing, and inventory. International models such as Estonia's e-Government and Singapore's Smart Nation initiative demonstrate the benefits of full digital integration.

This highlights the gap in Sri Lanka's e-Governance efforts and reinforces the need for a complete municipal management system for councils like Horana UC.

1.5 Aim

To design and implement a Smart Urban Council Management System for Horana UC using the MERN stack (MongoDB, Express.js, React.js, Node.js), with the goal of automating essential municipal functions and improving service delivery.

1.6 Objectives

The project is guided by the following objectives:

- Develop a centralized web platform to streamline council services.
- Provide citizens with the ability to submit and track complaints online.
- Implement permit and license processing with officer approvals.
- Enable secure online payments for property tax, shop rent, and permits.
- Automate HR and payroll management for council staff.
- Create multilingual interfaces to support Sinhala, Tamil, and English.
- Integrate an AI chatbot for bookings with conflict prevention checks.
- Provide real-time notifications and PDF license/report generation.

1.7 Solution Overview

The Horana Urban Council Smart Management System is a web-based application that integrates multiple council services into one platform. It uses:

- React.js for user-friendly, responsive front-end.
- Node.js + Express.js for backend APIs.
- MongoDB for scalable, flexible database management.
- Stripe Dummy Gateway for secure online payments.
- NodeMailer for email notifications.
- PDF generators for licenses, receipts, and reports.
- Chatbot integration for booking services (playgrounds, crematoriums).

1.8 Methodology

To successfully design and implement the Horana Urban Council Smart Management System, we followed a structured development methodology that ensured efficiency, collaboration, and flexibility throughout the project. Our approach includes five key phases: Requirements Engineering, System Design, Development, Testing, and Deployment all aligned under the Agile development methodology.

Agile Methodology Overview

Agile methodology is an iterative and incremental project management approach that focuses on flexibility, collaboration, and continuous improvement. Instead of delivering the entire system at the end of development, Agile encourages dividing the project into smaller, manageable parts (sprints). At the end of each sprint, a working product increment is delivered, tested, and reviewed.

Key Agile principles applied in this project include:

- Individuals and interactions over strict processes.
- Working software delivered early and frequently.
- Customer collaboration for feedback and validation.
- Responding to change rather than following a rigid plan.

This approach is well-suited for the Horana Urban Council Smart Management System because it is composed of multiple independent modules (complaints, payments, HR, bookings, shop registration) that can be developed and tested separately while still forming an integrated solution.

Why Agile Was Chosen for This System

The decision to use Agile methodology was based on the following project-specific factors:

- Multiple Modules with Different Priorities
The HUC system includes diverse modules such as citizen complaints, property tax, shop registration, bookings, HR/payroll, and finance reports. Agile allowed the team to prioritize and deliver the most critical modules (citizen complaints and property tax) first, while gradually adding more complex ones (HR/payroll, chatbot booking) later.
- Frequent Requirement Changes
During requirement gathering, feedback from council staff and mock citizens indicated evolving needs (e.g., multilingual support, chatbot booking). Agile provided the flexibility to adapt requirements during sprints without derailing the entire project plan.
- Continuous Testing and Feedback
At the end of each sprint, functional modules were tested, and improvements were made before moving forward. This reduced risks compared to a traditional waterfall model where issues are discovered only at the end.
- Collaboration Among Team Members
With members working on different modules (e.g., Chamya on property/payments, Amarajeewa on bookings, others on HR & admin), Agile provided a structured yet flexible way to coordinate development through daily stand-ups, GitHub commits, and sprint reviews.

Agile Phases Applied in the HUC Project

Requirement Gathering

- Conducted interviews with Horana UC officers and discussions with sample citizens.
- Identified pain points such as manual tax collection, no complaint tracking, and booking conflicts.

- Documented both functional requirements (e.g., “Citizens can pay property tax online”) and non-functional requirements (e.g., “System must be multilingual, secure, and available 24/7”).
- Tools used: Google Docs for documentation, GitHub Issues for requirement tracking.

System Design

- Created UML diagrams (use case diagrams for each module, activity diagrams for workflows).
- Designed Entity Relationship Diagram (ERD) for MongoDB collections (citizens, officers, properties, bookings, payments, employees).
- Built UI mockups in Figma for citizen dashboards, officer panels, and admin controls.
- Architectural planning: adopted MERN stack with modular APIs.

Development

Adopted 2-week sprints for structured progress.

Each sprint concluded with **internal review & bug fixing**.

Collaboration tools: GitHub for version control, Microsoft Teams & WhatsApp for communication.

Testing

- Unit Testing: Tested individual modules like tax calculation, booking forms.
- Integration Testing: Verified communication between modules (e.g., payment confirmation linked to shop registration approval).
- User Acceptance Testing (UAT): Collected feedback from sample users playing the roles of citizens and officers.
- Performance Testing: Ensured response times were fast enough for multiple simultaneous users.

Deployment

Final system was deployed on a cloud platform (Heroku/Render) to ensure 24/7 accessibility. Citizens could access via browser, while officers and admins had role-based dashboards. Security ensured via encrypted passwords and secure API endpoints.

Sprint 1: Core System Development

Deliverables:

- Citizen complaint submission
- Permit/License online application
- Assessment addition & secure online payments
- Employee salary slip viewing
- Inventory item addition

Sprint 2: Officer & HR Workflows

Deliverables:

- Complaint status update
- Officer permit approval/rejection
- Assessment update & payment status tracking
- HR salary management & adjustments
- Inventory stock updates

Sprint 3: Reports & Advanced Features

Deliverables:

- Duplicate complaint removal
- Citizen permit tracking & permit reports
- Assessment reports & payment receipts
- Payroll logs and reports
- Inventory reports & low-stock alerts

Benefits Realized Through Agile

- **Reduced Development Risks:** Problems were identified early during sprint reviews.
- **Improved Collaboration:** Tasks assigned per sprint, ensuring everyone contributed to meaningful progress.
- **Higher Quality Output:** Continuous testing after each sprint ensured fewer bugs in the final product.
- **User-Centric Development:** Mock feedback loops helped refine features (e.g., adding multilingual options).

1.9 The Structure of the Report

Section 1:

The first section of the report covers the client's problems and the present state of the manual system. As a solution, it describes what the system should support and what the customer expects in terms of deliverables.

Section 2:

Requirement Analysis, Design, Implementation, and Testing are all covered in part two of the report. The Requirements Analysis is presented using use case diagrams and activity diagrams. The general design of the system is represented by the ER diagram, Class diagram, and Interface diagrams. The implementation specifies the system's module architecture, workflows, and test cases.

Section 3:

The third part contains the evaluation of the system, conclusion, and references that were utilized to help prepare the project report. Appendices are also included, containing member contributions, algorithms, and supporting documents.

Chapter 2. Requirements

2.1 Stakeholder analysis

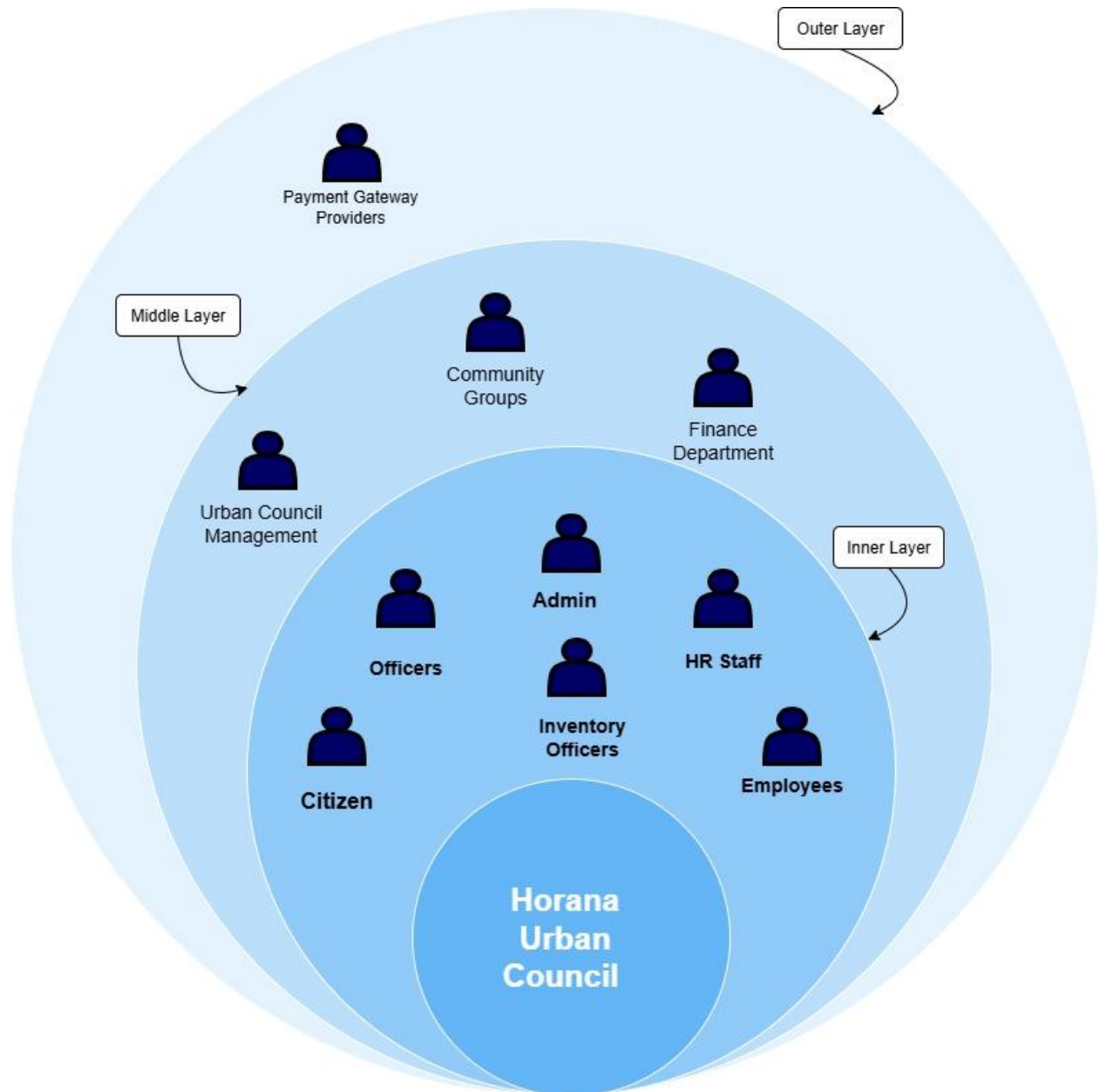


Figure 1- Onion Diagram

Horana Urban Council operations, the following stakeholder groups were identified:

- **Citizens:** The primary users who will interact with the system to submit complaints, apply for permits and licenses, make online payments, and book services such as playgrounds or crematoriums. Their expectations are convenience, transparency, multilingual support (Sinhala, Tamil, English), and access to services 24/7.
- **Council Service Officers:** Staff members responsible for processing and managing citizen applications, complaints, and bookings. They require dashboards that streamline workflows, prevent duplicate records, and allow approvals or rejections with justifications.
- **Finance Officers:** Officials tasked with revenue collection, financial reporting, and auditing. They expect automation of tax calculations, fines, and rental payments, together with the generation of income and expense reports in printable formats.
- **HR and Employees:** Council employees and HR staff who require a digital mechanism to manage employee records, salary slips, leave, and overtime. Automation is needed to reduce errors and improve

2.2 Requirements Analysis

2.2.1 Functional Requirements

The system's functional requirements are categorized by modules:

1. Citizen Services

- Submit complaints online and receive a unique complaint ID.
- Track complaints through statuses: *Pending* → *In-progress* → *Resolved*.
- Apply for shop registrations and permits with document upload.
- View approval/rejection decisions with reasons provided by officers.
- Book playgrounds or crematorium slots via form or chatbot.
- Make online payments for property taxes, shop rents, and other services.
- Download receipts and temporary licenses in PDF format.
- Receive email and dashboard notifications on status updates.

2. Officer Services

- Access a dashboard of citizen applications and complaints.
- Approve or reject requests with remarks.
- View booking calendars to prevent duplicate scheduling.
- Generate PDF reports on bookings, complaints, and registrations.

3. Finance and Payment Services

- Auto-calculate annual and quarterly property tax.
- Add late-payment fines automatically.
- Provide a secure payment gateway integrated with receipts.
- Generate revenue reports for council management.

4. HR and Payroll

- Manage employee records, leave, and overtime.
- Automatically calculate salaries, deductions, and allowances.
- Generate monthly pay slips for employees.

5. Administration

- Manage user roles and access rights (citizen, officer, finance officer, HR staff, admin).
- Monitor system performance and maintain backups.

2.2.2 Non-Functional Requirements

In addition to functionality, the system must satisfy quality attributes:

- Usability: Provide a simple and intuitive interface; support Sinhala, Tamil, and English; mobile-responsive design.
- Availability: Ensure 24/7 system availability through cloud hosting; provide redundancy and backups.
- Performance: Handle 500+ concurrent users; ensure page load times under three seconds.
- Security: Use encrypted password storage, secure payment gateways, and role-based access control.
- Scalability: Allow expansion to other urban councils and support additional modules like inventory management.
- Maintainability: Employ modular MERN architecture and GitHub version control for easier upgrades.
- Transparency: Provide citizens with real-time updates and councils with evidence-based reports.

2.3 Requirements Modeling

2.3.2 Activity Workflows

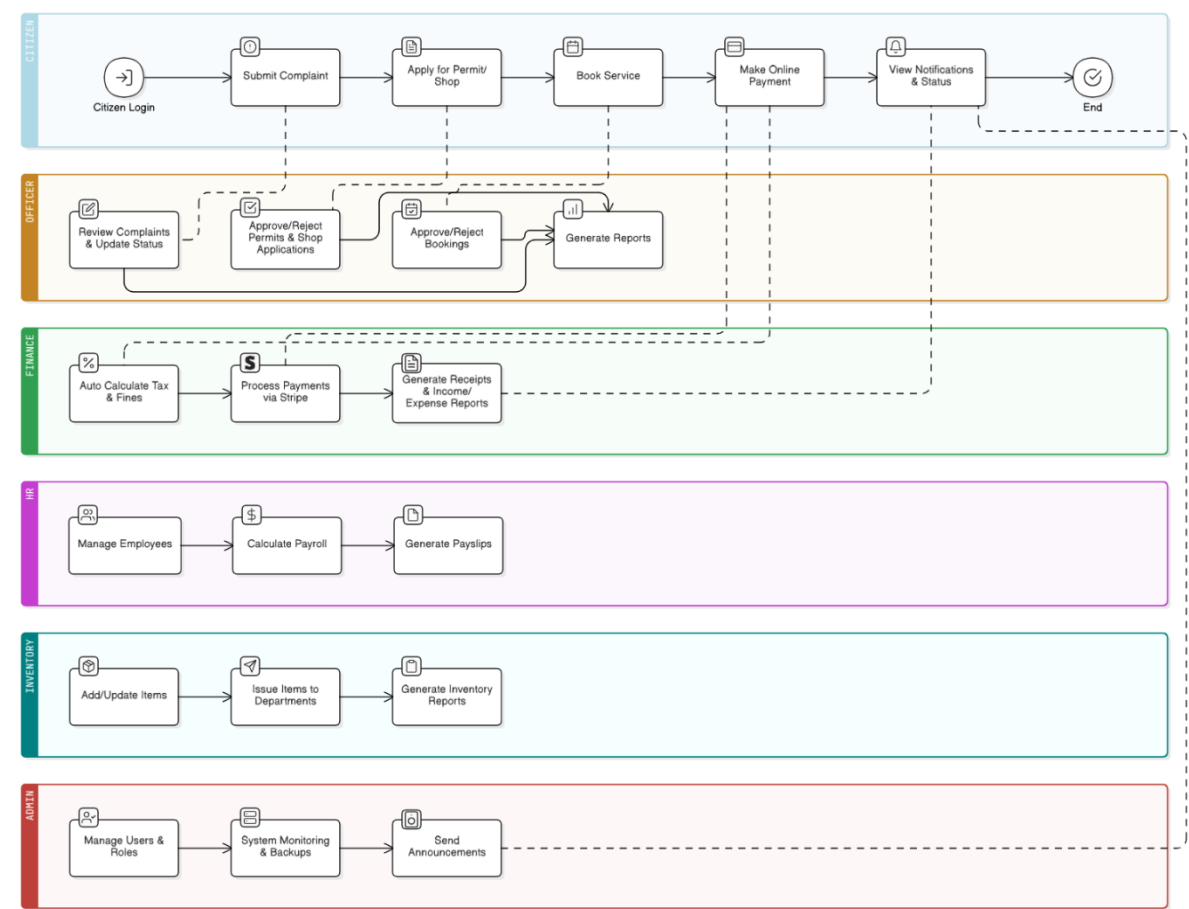


Figure 2- System Flow Diagram

2.3.3 Data Model

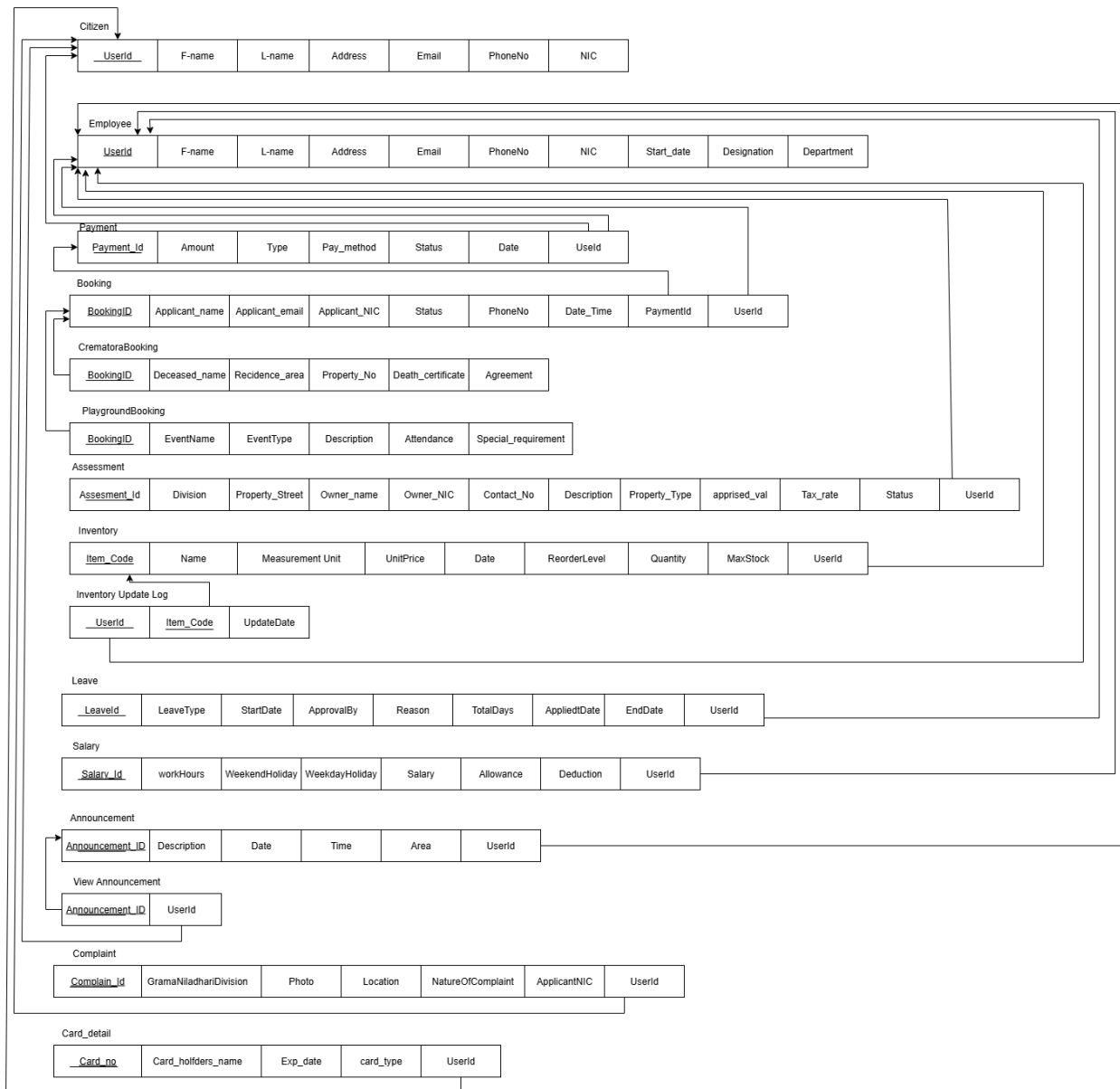


Figure 3-Database Schema

2.4 Technical Requirements

The proposed system requires both **hardware** and **software resources** to ensure smooth development, deployment, and usage. Technical requirements are classified under client-side (citizen access), server-side (council and hosting), and development environment.

2.4.1 Software Requirements

Client Side (Users – Citizens & Officers):

- Browser: Google Chrome, Mozilla Firefox, Safari, or Microsoft Edge (latest versions)

- OS: Windows, macOS, Linux, Android 10, iOS

Server Side (Hosting & Backend):

- Operating System: Ubuntu Server 20.04 LTS (preferred)
- Backend: Node.js v18+ with Express.js
- Frontend: React.js v18+
- Database: MongoDB 6.0 (NoSQL, document-oriented)
- Payment Integration: Stripe (sandbox/test gateway)
- PDF Generation: jsPDF or PDFKit
- Email Service: NodeMailer (SMTP integration)
- Chatbot Integration: Dialogflow / Botpress

Development Environment:

- IDE/Editor: Visual Studio Code
- Version Control: Git + GitHub Repository
- Project Management: Trello / Jira for sprint tracking
- Design Tools: Figma (UI mockups), Draw.io / Lucid chart (UML diagrams)
- Testing Tools: Postman (API testing)

2.4.2 Network Requirements

- Client-Server Architecture: Web-based access via HTTPS.
- Secure Communication: TLS/SSL certificates for encrypted transactions.
- Authentication: Role-based access (JWT tokens).
- Hosting: Cloud-based (Heroku, Render, or AWS EC2).
- Database Hosting: MongoDB Atlas (cloud-managed database)

2.4.3 Security Requirements

- Role-based access ensures citizens, officers, admins, and HR have controlled permissions.
- Secure payment via Stripe API sandbox.
- Regular system backups for data recovery.

Chapter 3 – Design and Development

3.1 System Architecture

The system follows a **three-tier architecture**:

1. Presentation Layer (Frontend)

- Developed with React.js for a dynamic and responsive user interface.
- Citizens, officers, HR staff, and administrators each interact with role-specific dashboards.
- User inputs are validated before being sent to the backend.

2. Application Layer (Backend)

- Implemented with Node.js and Express.js.
- Handles business logic such as tax calculation, booking conflict detection, payroll processing, and complaint routing.
- Provides RESTful APIs that connect the frontend to the database.

3. Data Layer (Database)

- MongoDB used as the document-oriented NoSQL database.
- Stores citizens, complaints, permits, bookings, payments, employee records, and inventory.
- Enables scalability and fast retrieval for large volumes of records

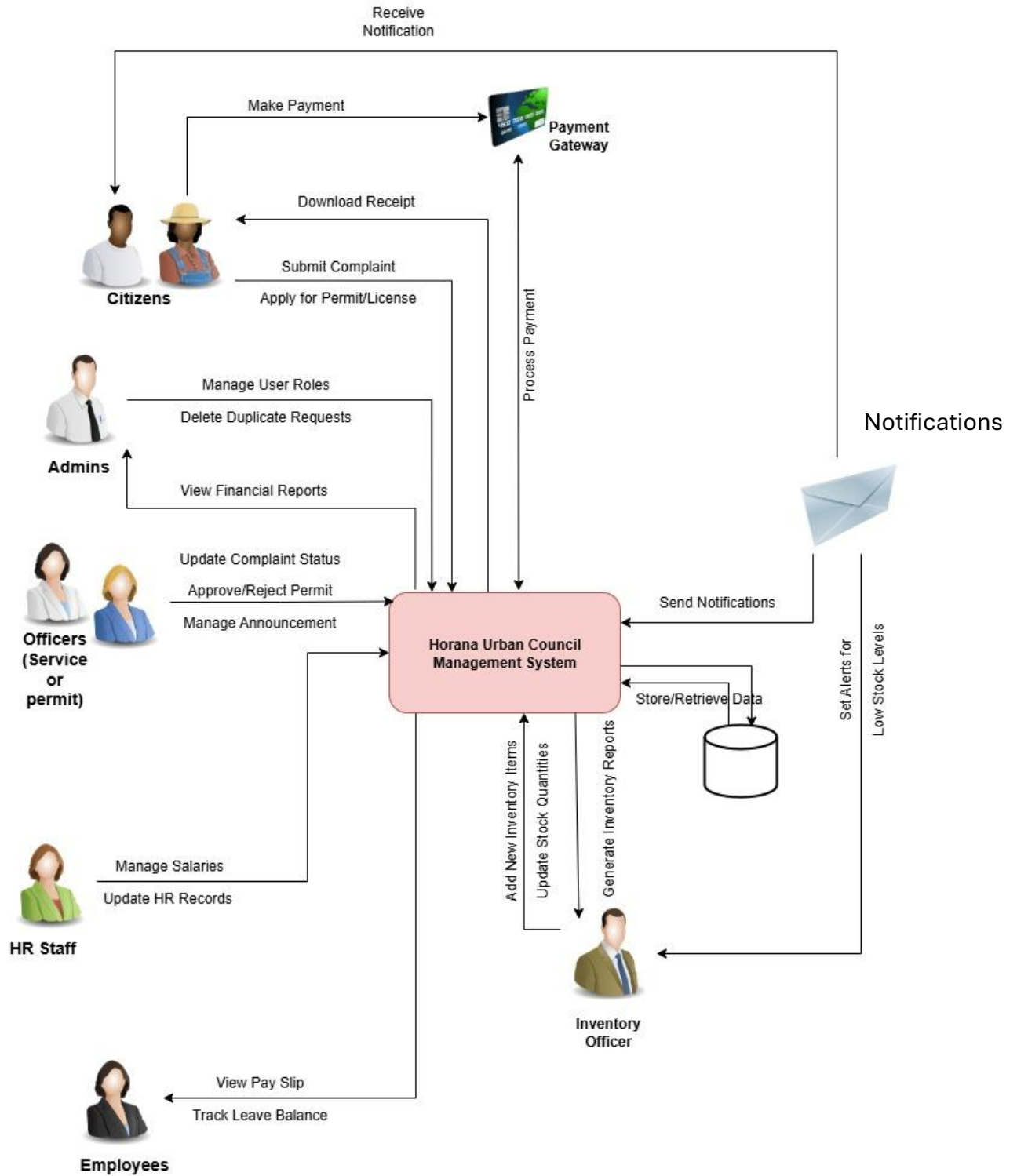


Figure 4- System Diagram

3.2 Diagrams of Components

3.2.1 Kavishka S P P (IT23815582)- Complaint Management Module

- **Actors:** Citizens, Officers
- **Functions:**
 - Citizen submits complaint (web form).
 - System assigns Complaint ID and sets status as *Pending*.
 - Officer reviews, updates status, adds notes.
 - Citizen receives updates by email/dashboard.
- **Design Considerations:**
 - Simple UI for citizens.
 - Officer dashboard with filtering by complaint category/status.

Use Case Diagram for Complaint

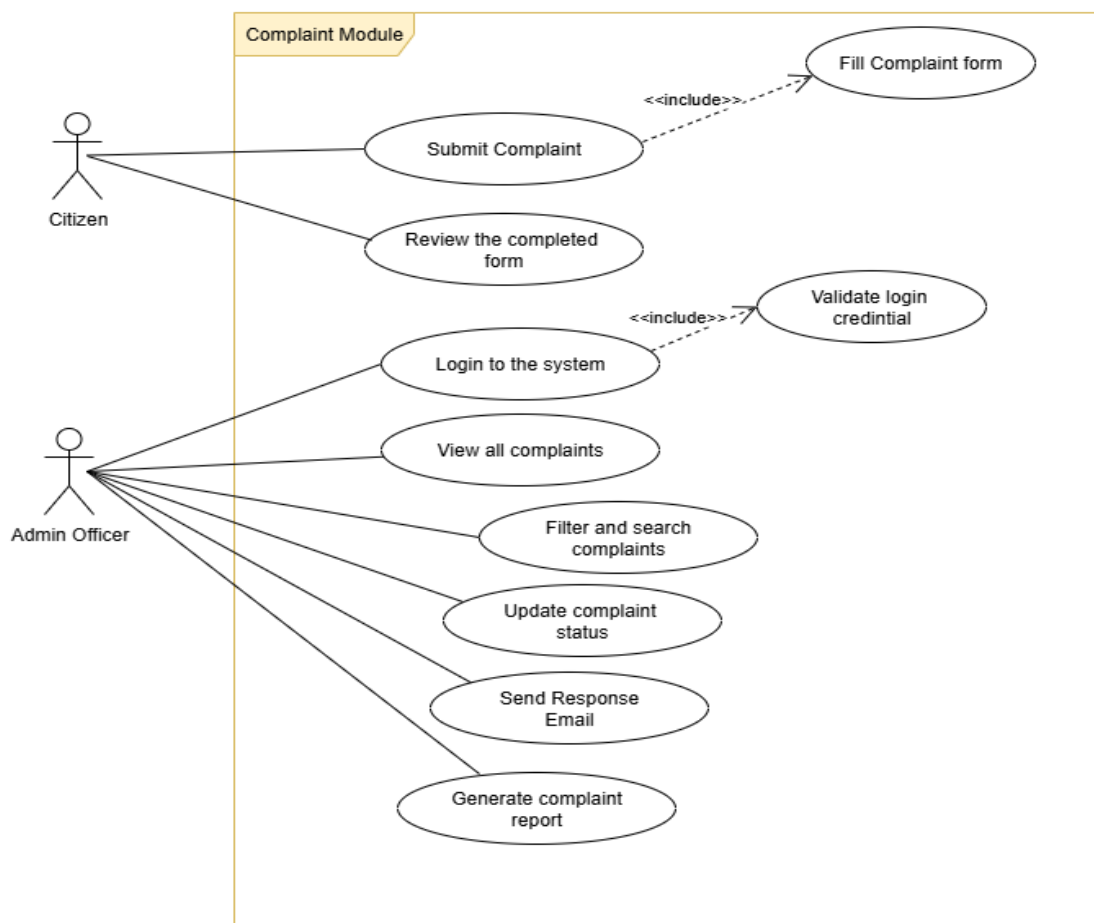


Figure 5- Use Case Diagram for Complaint

Sequence Diagram for submit complaint

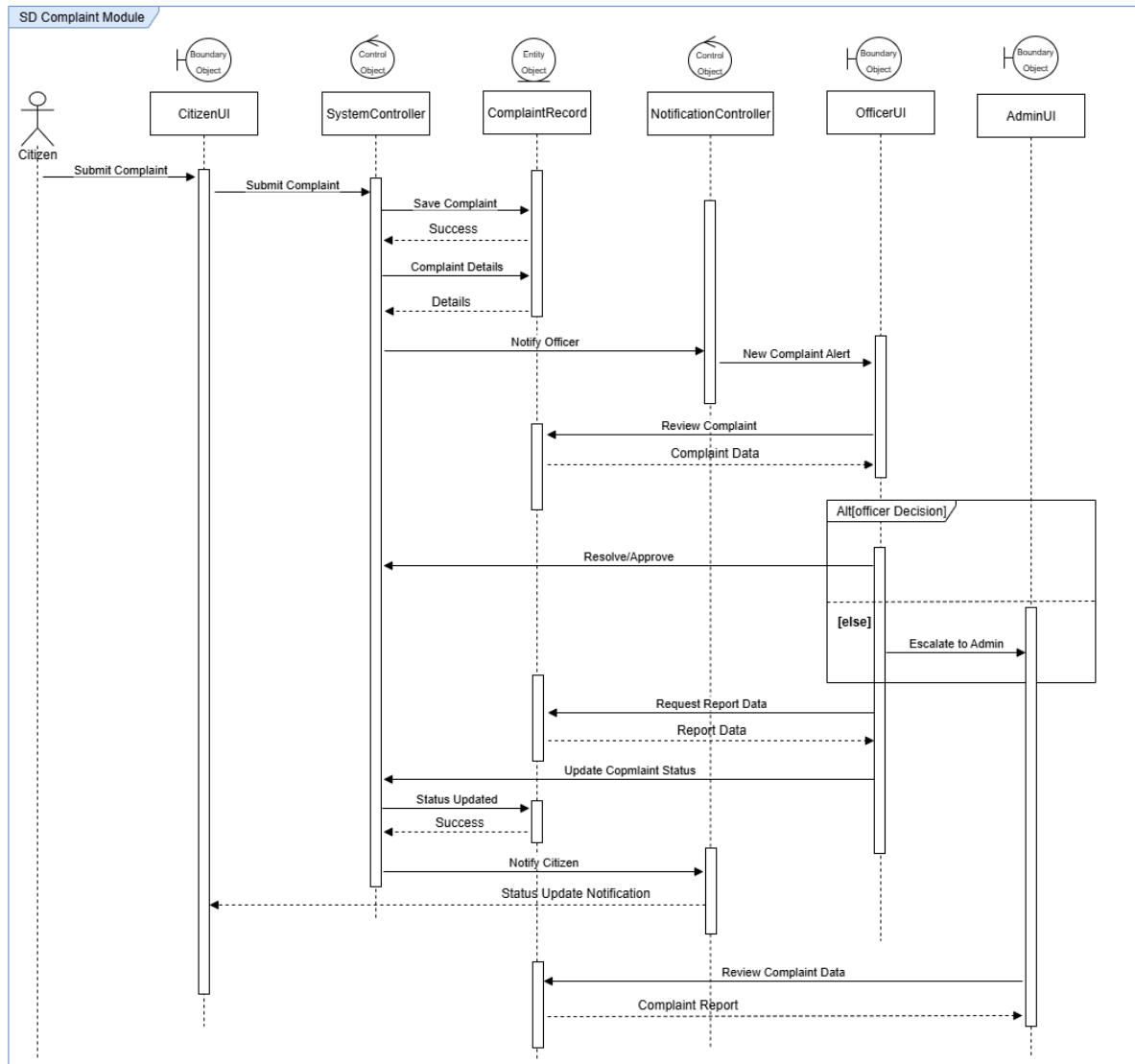


Figure 6- Sequence Diagram for Submit Complaint

Activity diagram for submit complaint

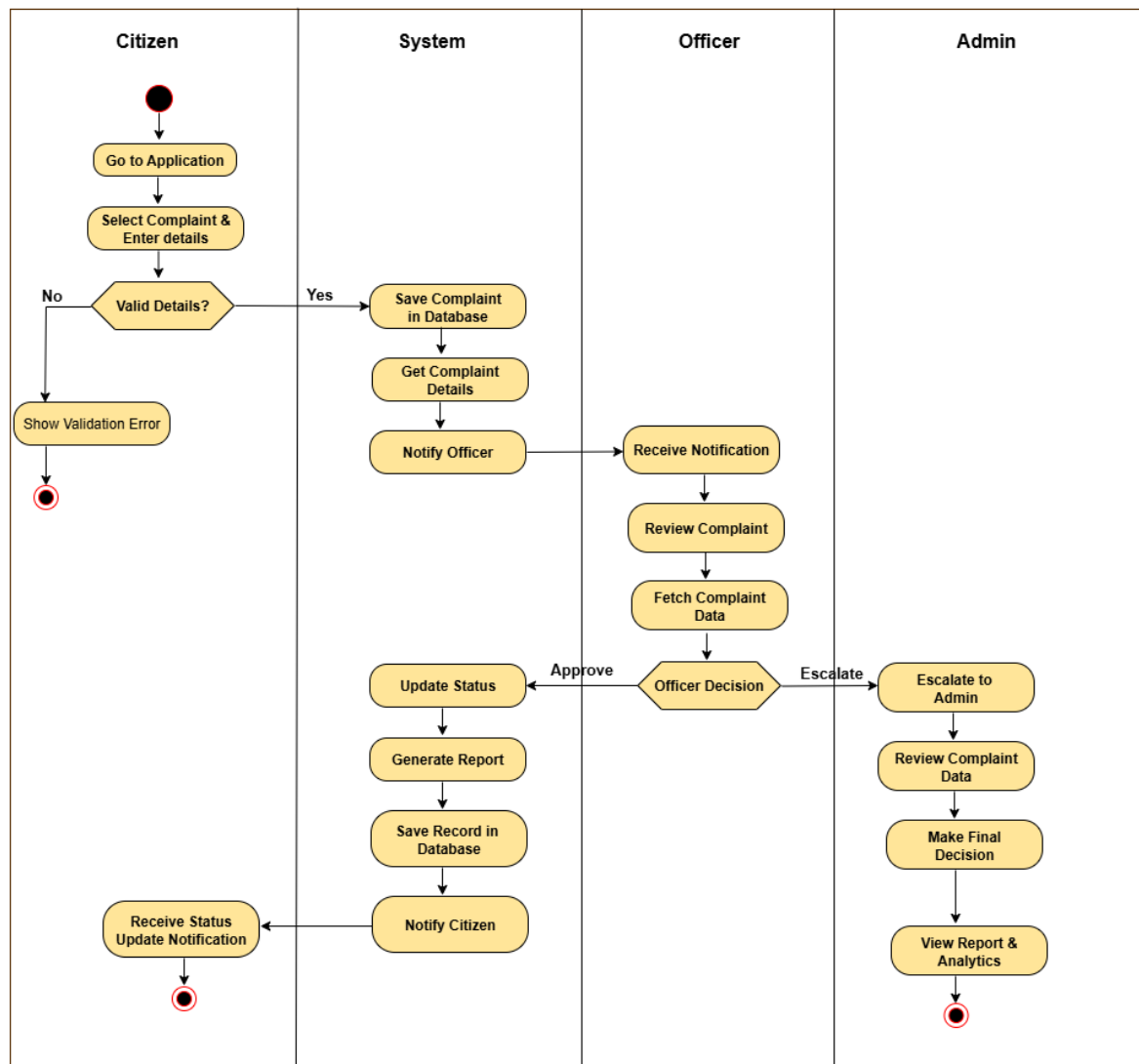
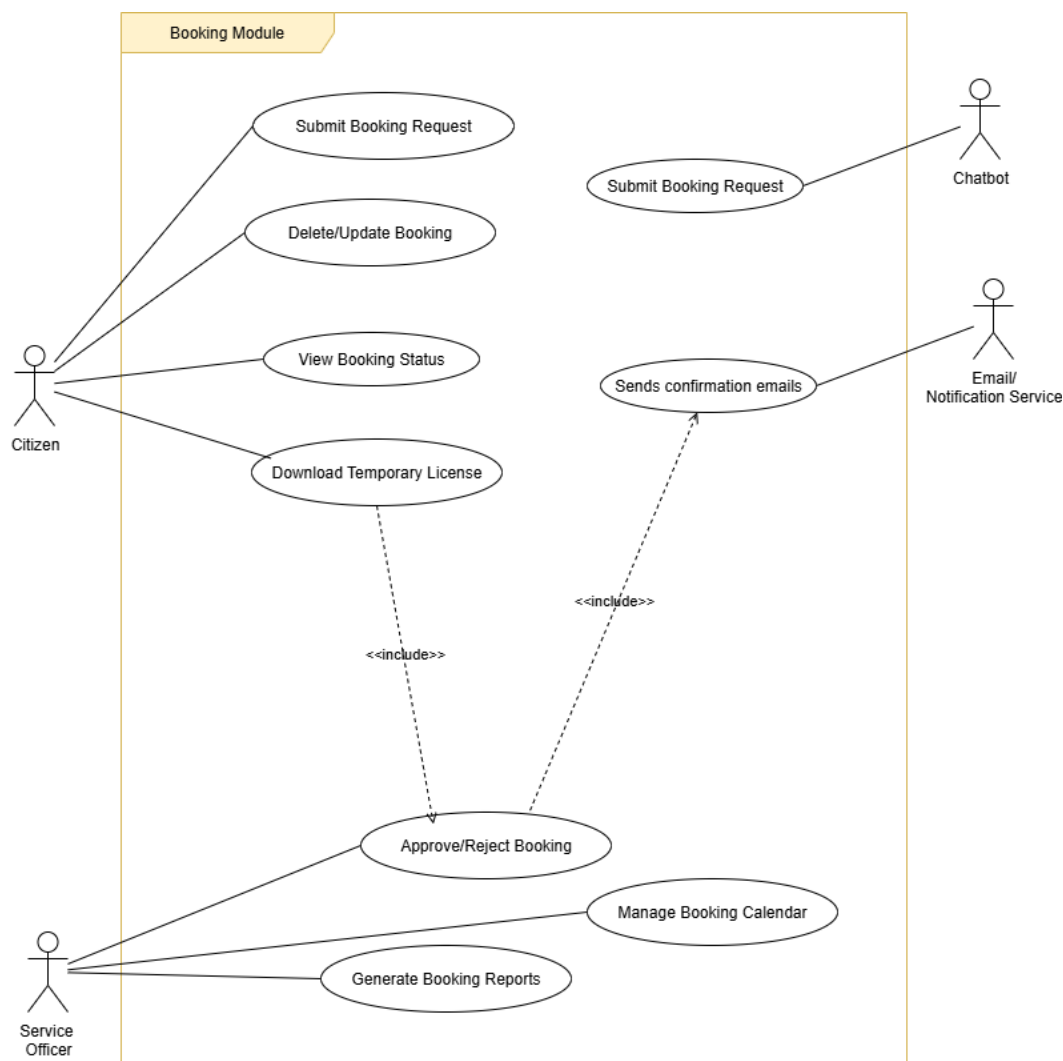


Figure 7- Activity diagram for Submit Complaint

3.2.2 Amarajeewa S D A S (IT23817562) Service Booking Module (Playground & Crematorium)

- **Actors:** Citizens, Officers
- **Functions:**
 - Citizens book time slots via web or chatbot.
 - System checks for duplicate/conflicting bookings.
 - Officer approves/rejects booking.
 - PDF confirmation license generated on approval.
- **Design Considerations:**
 - Conflict prevention algorithm.
 - Calendar view for officers.

Use Case Diagram for booking service*Figure 8- Use case Diagram for Service*

Sequence Diagram for booking a service

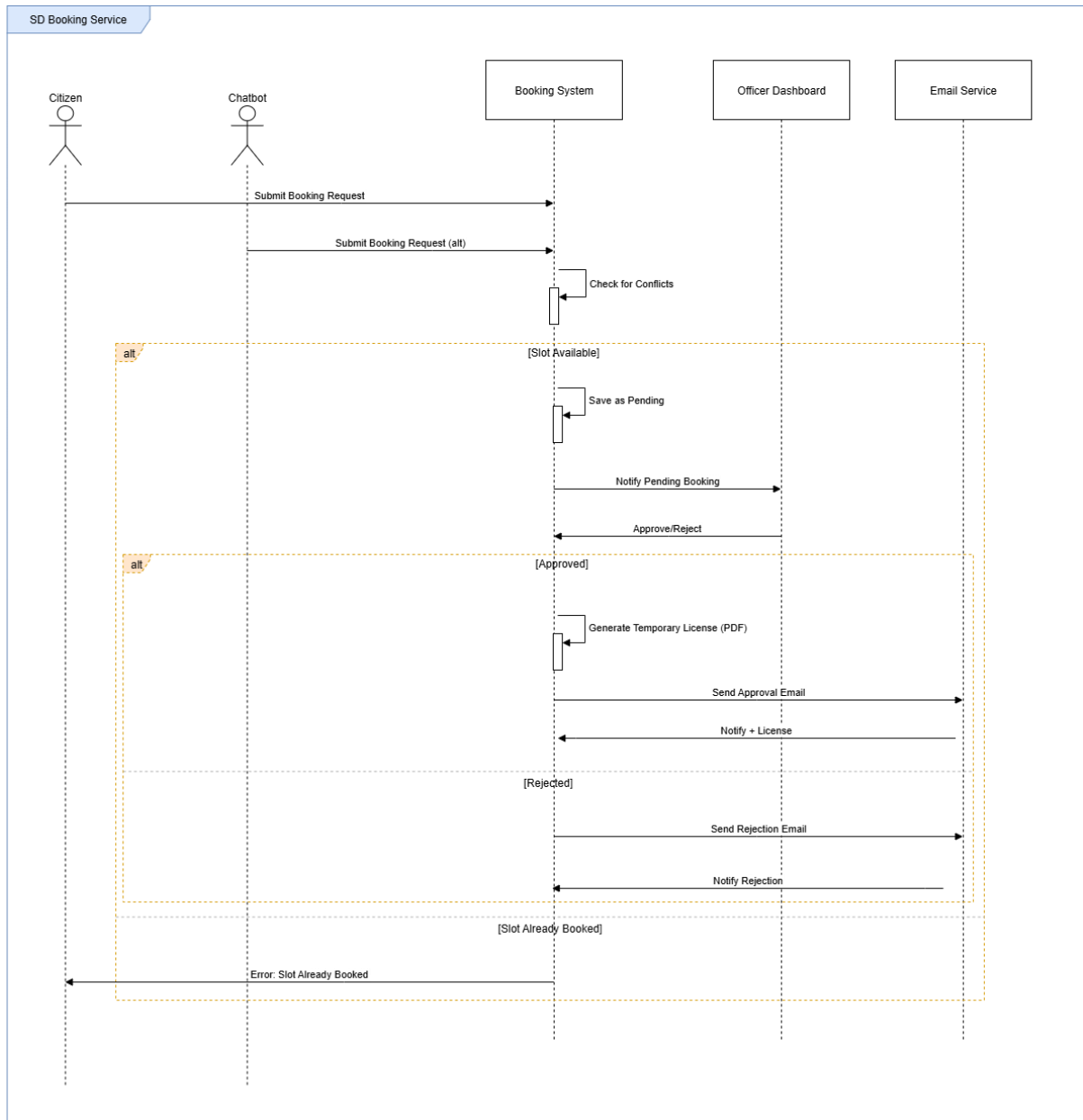


Figure 9- Sequence diagram for Booking sytem

Activity Diagram for booking a service

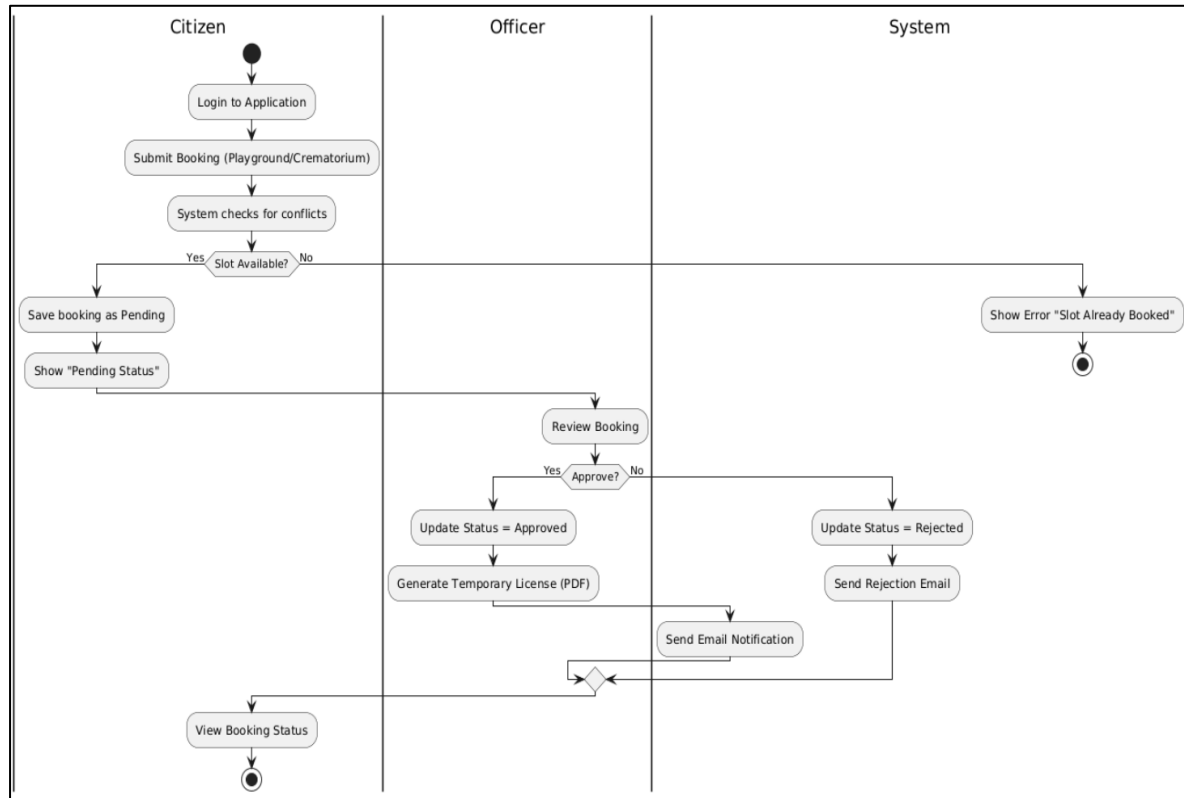


Figure 10- Activity diagram for Booking a service

3.2.3 Chamya N D (IT23848184) Online Payments & Finance Module

- **Actors:** Citizens, Finance Officers
- **Functions:**
 - Citizens pay property tax, shop rent, or booking fees online.
 - Stripe (sandbox) processes payments.
 - System generates receipts in PDF.
 - Finance officers access dashboards with income/expense reports.
- **Design Considerations:**
 - Auto-tax calculation and late fine integration.
 - Secure API for payment handling.

Use Case Diagram for Payment Module



Figure 11- Use Case Diagram for Payment Module

Sequence Diagram for Payment Module

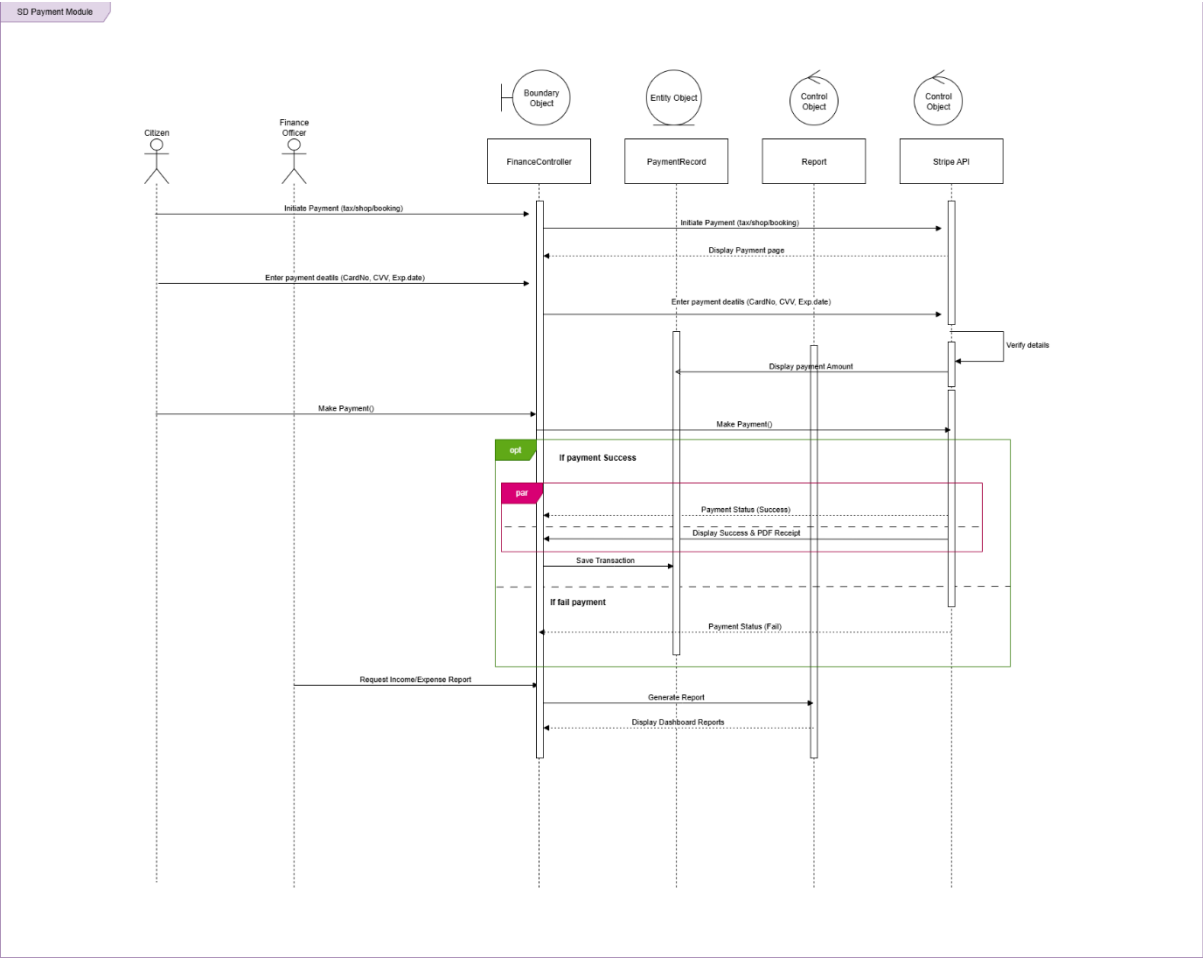


Figure 12- Sequence Diagram for Payment Module

Activity diagram for Payment Module

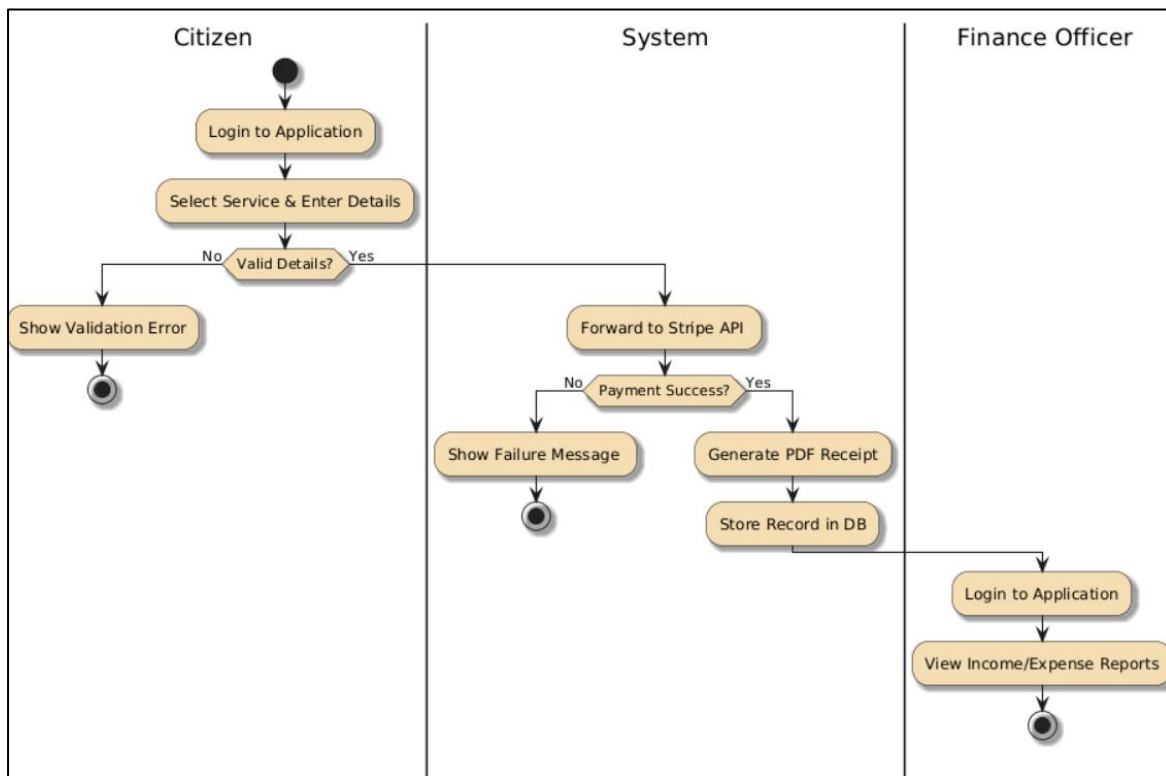


Figure 13- Activity diagram for Payment Module

Activity diagram for Shop Property Application

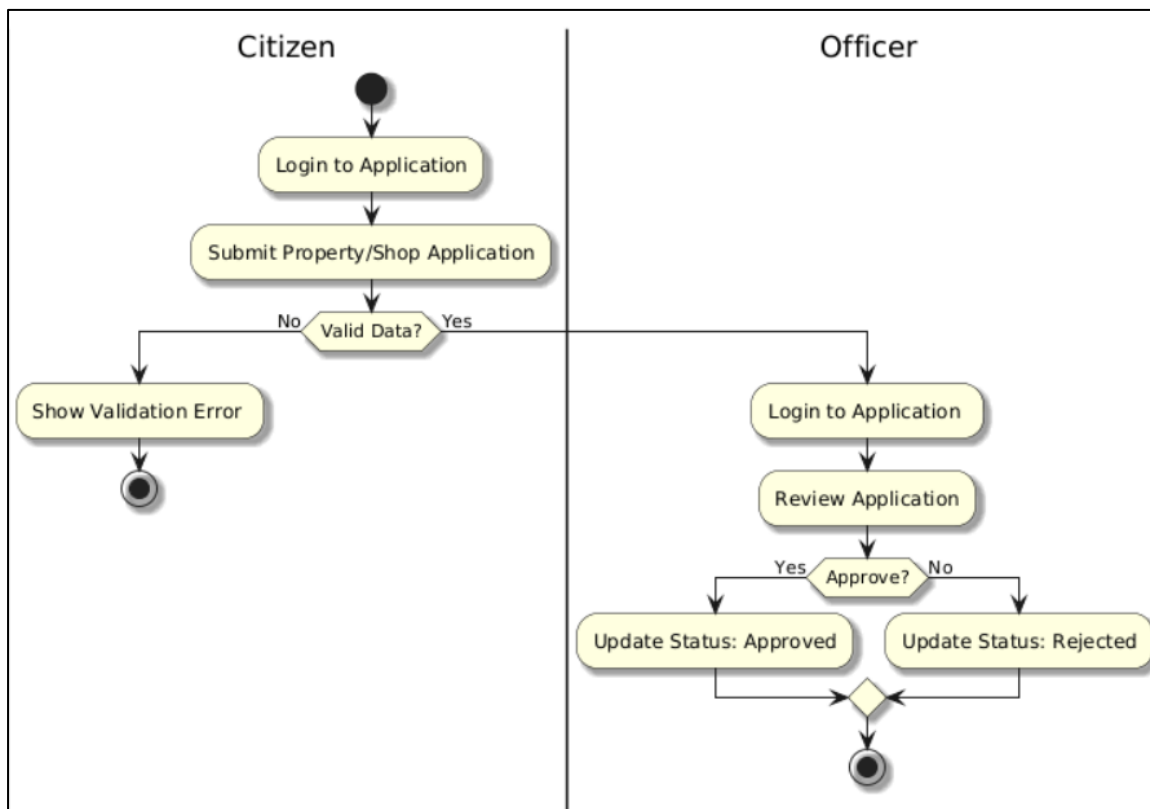


Figure 14- Activity diagram for Shop Property Application

3.2.4 HR & Payroll Module

- **Actors:** HR Officers, Employees
- **Functions:**
 - HR staff manage employee profiles.
 - Leave, overtime, and attendance tracked digitally.
 - Salaries auto-calculated; payslips generated.
 - Employees can log in to view payslips.
- **Design Considerations:**
 - Salary formula (base + OT – deductions).
 - Confidentiality through role-based access.

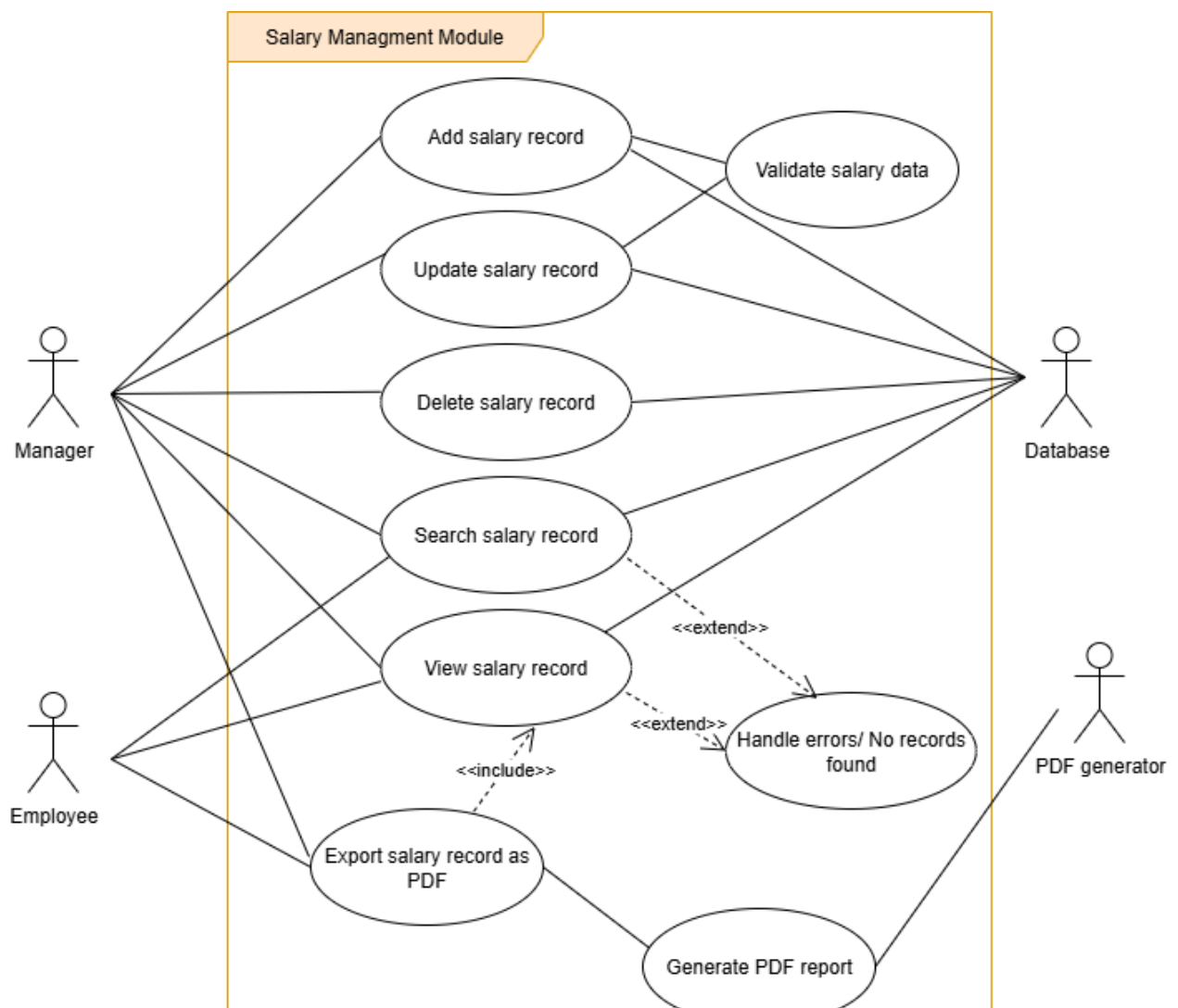
Use Case Diagram for salary

Figure 15- Use Case Diagram for salary

Sequence Diagram for salary calculation

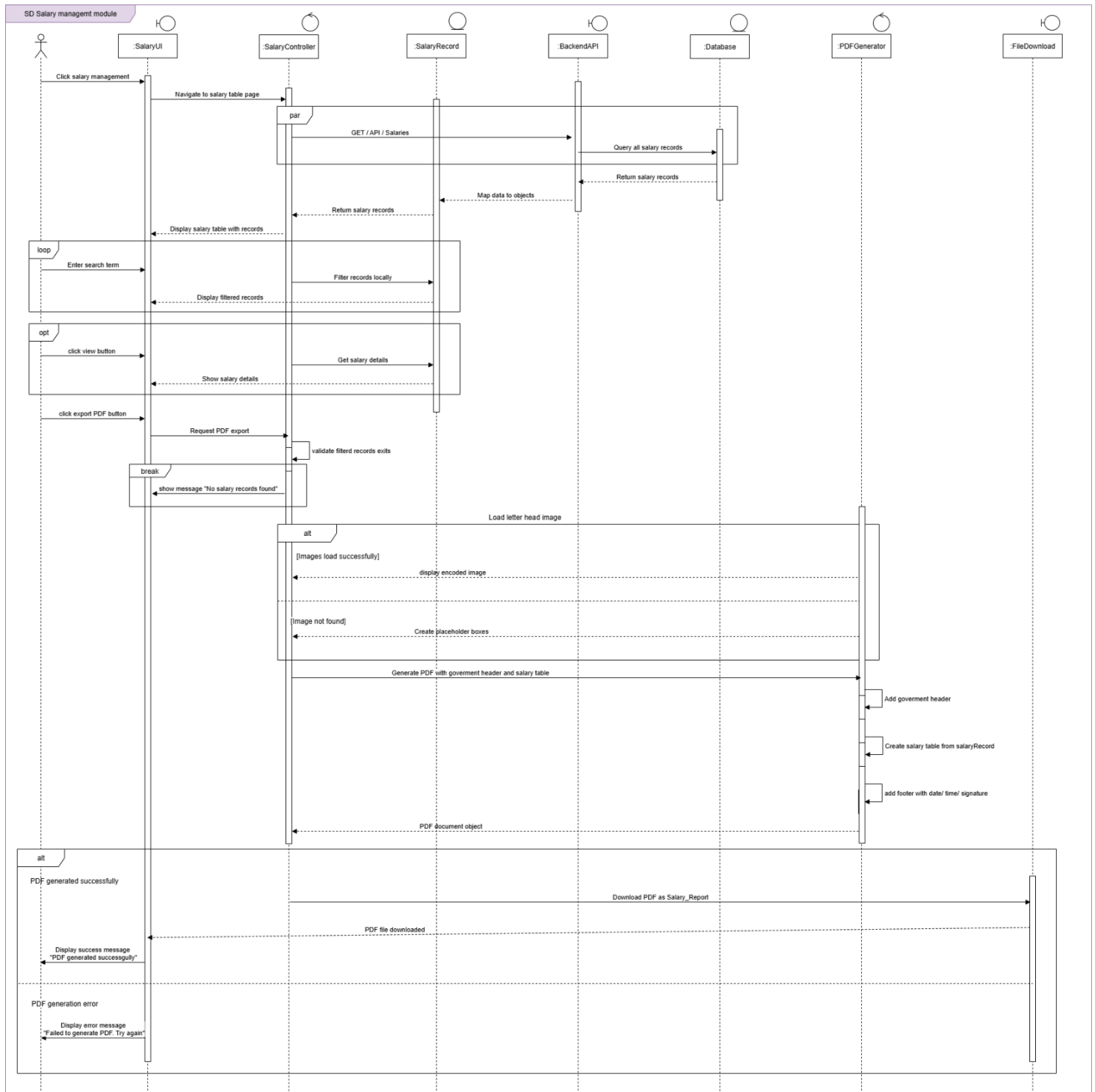
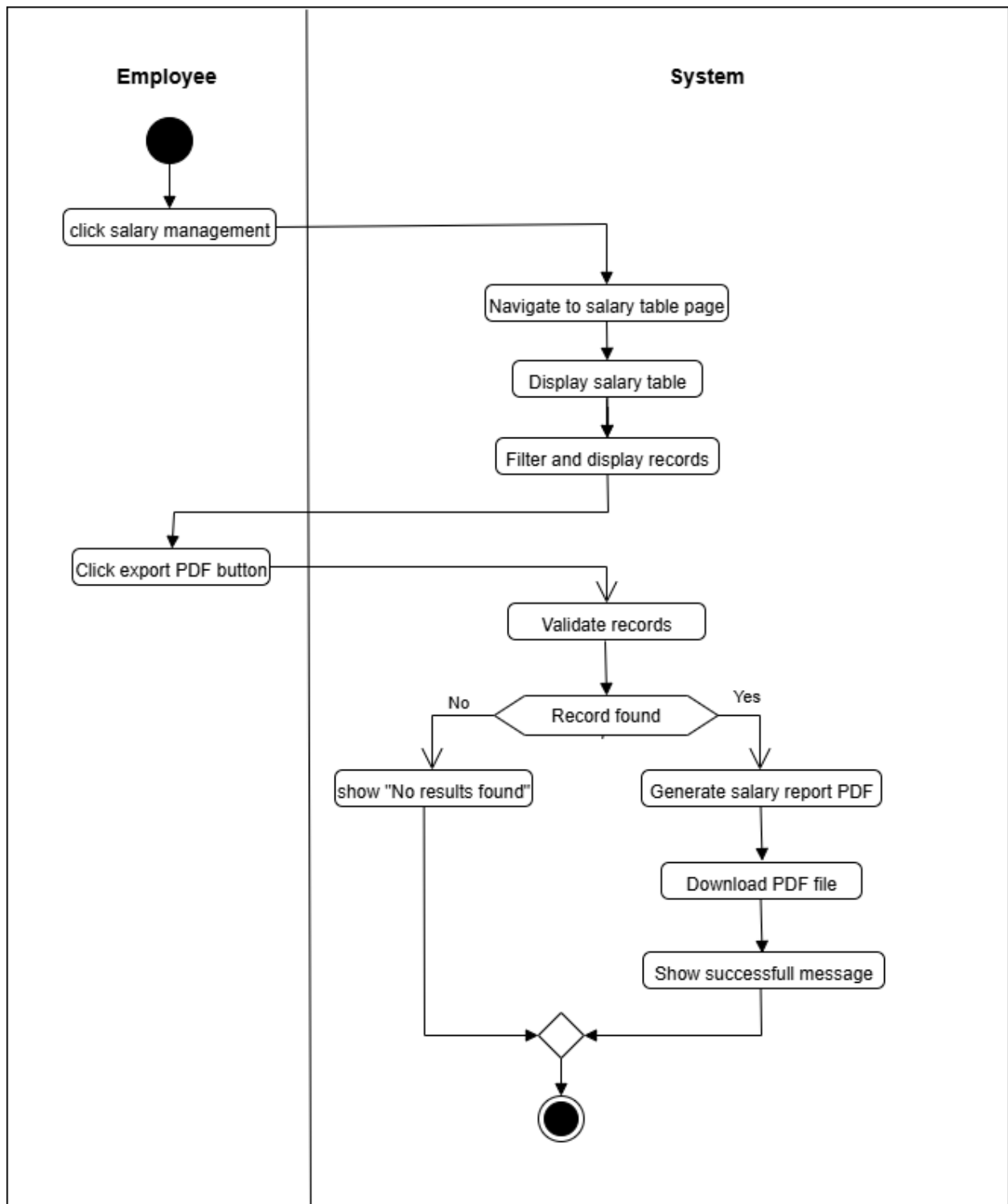
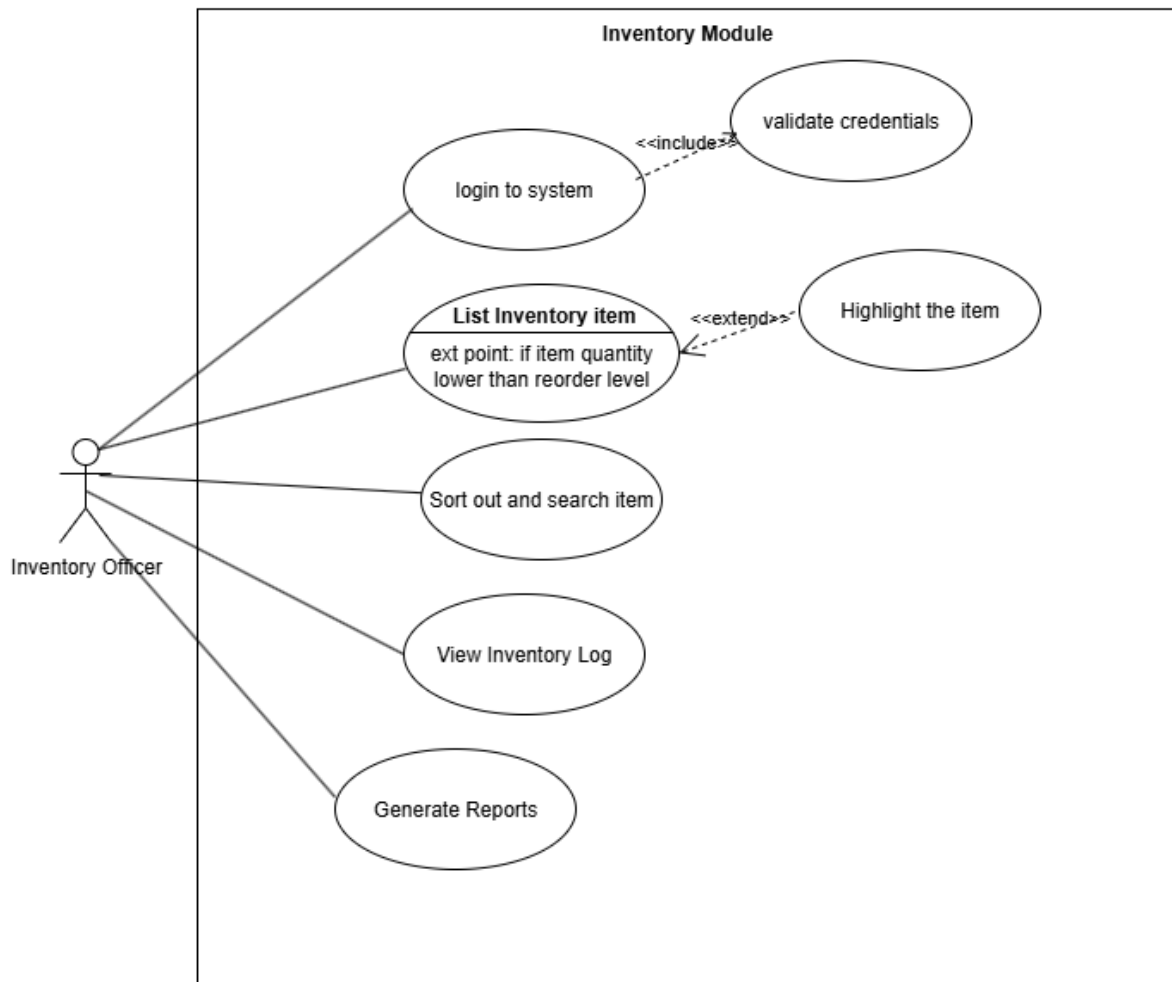


Figure 16- Sequence Diagram for salary calculation

Activity Diagram for salary management*Figure 17- Activity Diagram for salary management*

3.2.5 Inventory Management Module

- **Actors:** Inventory Officer, Admin
- **Functions:**
 - Add/update stock (chairs, IT equipment, books, office supplies).
 - Issue items to departments; update stock levels.
 - Mark out-of-stock items; reorder alerts.
 - Generate monthly inventory reports.
- **Design Considerations:**
 - Stock control through quantity thresholds.
 - Integration with Finance for purchase tracking.

Use Case Diagram for inventory*Figure 18- Use Case Diagram for inventory*

Sequence Diagram for add inventory item

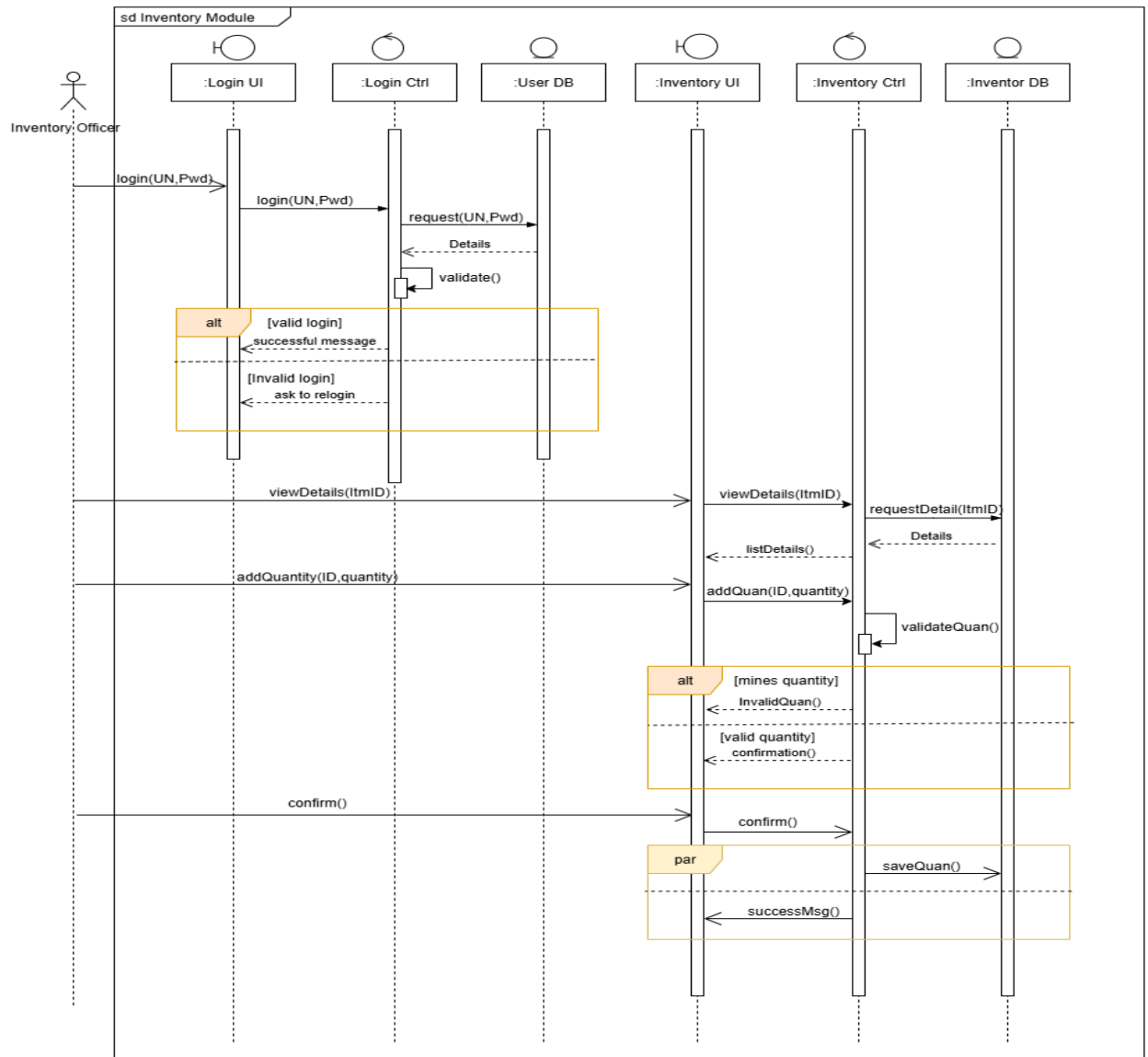


Figure 19- Sequence Diagram for add inventory item

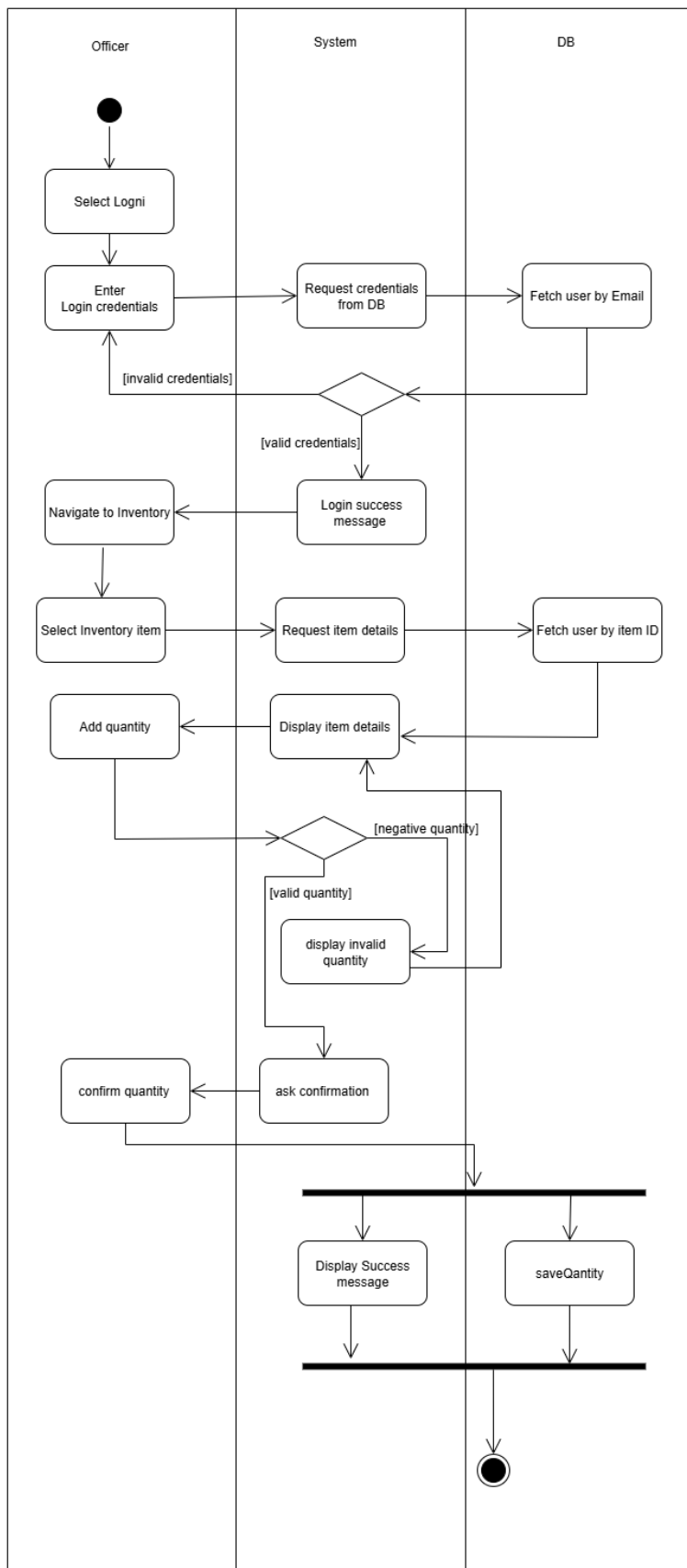
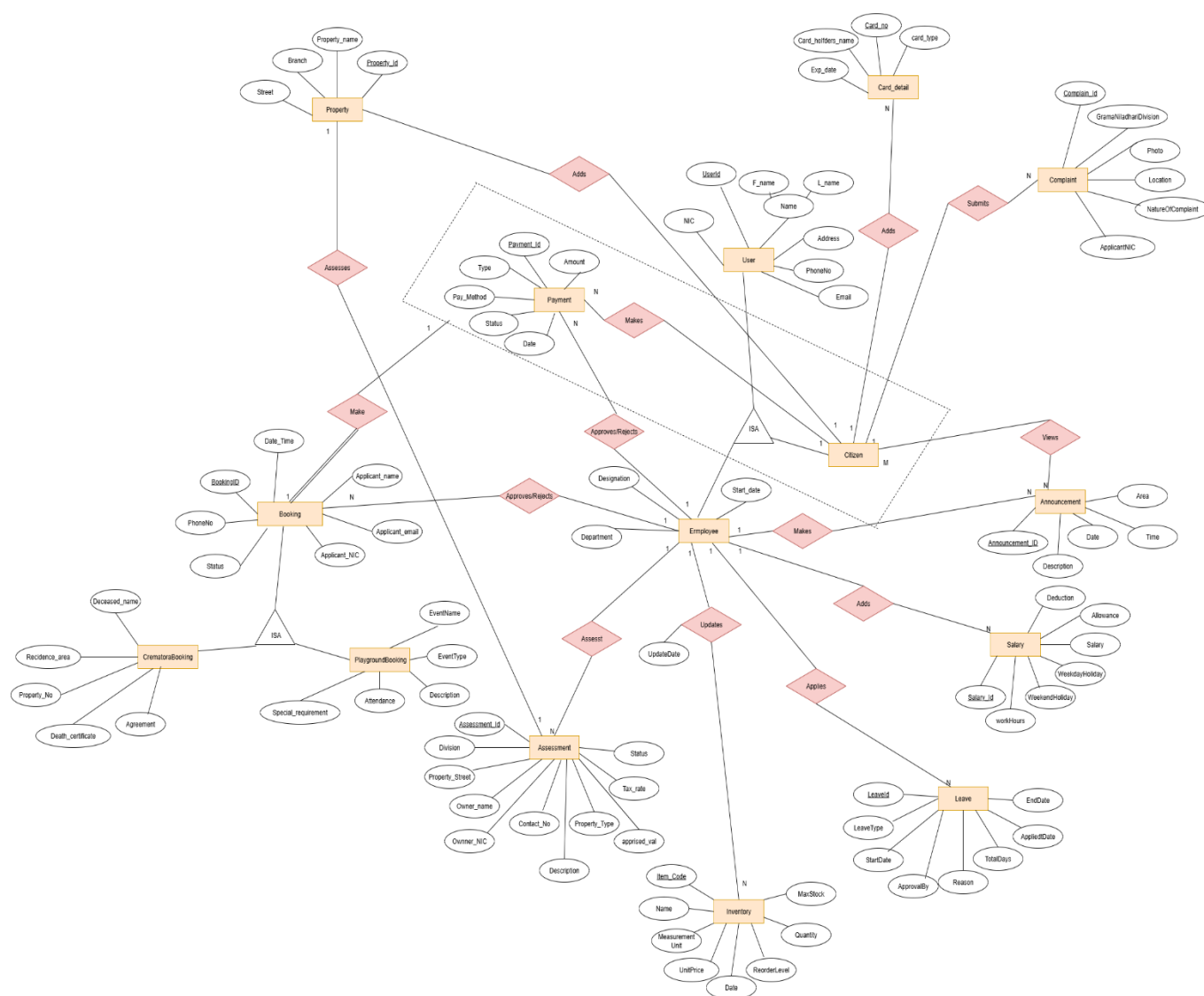
Activity Diagram for add inventory item

Figure 20- Activity Diagram for add inventory item

3.4 Database Design

ER Diagram



3.5 User Interface Design

Screen shots

- **Citizen Dashboard:** Submit/view complaints, apply permits, make payments, track bookings.
- **Officer Dashboard:** View applications, approve/reject, manage bookings, generate reports.
- **HR Dashboard:** Manage employees, payroll, payslips.
- **Admin Dashboard:** User management, role assignment, logs.

(Insert screenshots or UI mockups.)

3.6 Algorithms

- **Auto Tax Calculation Algorithm**

$\text{tax} = (\text{property_value} * \text{rate}) / 100$

if $\text{payment_date} > \text{due_date}$:

$\text{fine} = \text{tax} * 0.05$

$\text{total} = \text{tax} + \text{fine}$

- **Booking Conflict Prevention Algorithm**

if requested_slot in booked_slots :

$\text{reject_booking}()$

else:

$\text{save_booking}(\text{status}="Pending")$

- **Payroll Calculation Algorithm**

$\text{salary} = \text{base_salary} + (\text{OT_hours} * \text{OT_rate}) - \text{deductions}$

$\text{generate_payslip}(\text{employee_id}, \text{salary})$

- **Inventory Stock Management Algorithm**

if $\text{stock_quantity} < \text{reorder_level}$:

$\text{trigger_reorder_alert}()$

else:

$\text{issue_item}()$

$\text{update_stock}()$

Chapter 4 – Implementation

4.1 Technology Stack

Frontend (Client Side)

- **React.js v18** for interactive dashboards.
- **Bootstrap/Material UI** for styling and responsive layouts.
- **Axios** for API communication.

Backend (Server Side)

- **Node.js v18** runtime.
- **Express.js** framework for REST APIs.
- **JWT (JSON Web Tokens)** for authentication and role-based access.

Database

- **MongoDB Atlas** (cloud-hosted NoSQL database).
- Collections: Citizens, Complaints, Permits, Bookings, Payments, Employees, Inventory.

Third-Party Integrations

- **Stripe API (sandbox)** for payment processing.
- **NodeMailer** for email notifications.
- **jsPDF** for generating licenses, receipts, and reports.
- **Dialogflow** for chatbot-based booking.

4.2 Module Implementations

4.2.1 Complaint Management Module

- **Frontend:**
 - Citizen submits complaint through React form.
 - Complaint ID displayed upon submission.

- **Backend:**

```
// Complaint submission endpoint
app.post("/api/complaints", async (req, res) => {
  const complaint = new Complaint({
    citizen_id: req.body.citizen_id,
```

```

    category: req.body.category,
    description: req.body.description,
    status: "Pending"
  });
  await complaint.save();
  res.json({ message: "Complaint submitted", id: complaint._id });
});

```

- **Database:**

- complaint_id, citizen_id, category, description, status, date_created.

- **Integration:**

- Officer dashboard fetches complaints → updates status.
- Citizen notified via email.

4.2.2 Permit & License Module

- **Frontend:** Form for shop/business license with document upload.

- **Backend:**

```

app.post("/api/permits", async (req, res) => {
  const permit = new Permit({
    citizen_id: req.body.citizen_id,
    type: req.body.type,
    documents: req.body.documents,
    status: "Pending"
  });
  await permit.save();
  res.json({ message: "Permit applied", id: permit._id });
});

```

- **Integration:**

- Officer approves/rejects → system generates **PDF license**.

4.2.3 Service Booking Module

- **Conflict Prevention Algorithm:** Before saving booking, check slot availability.

```
app.post("/api/bookings", async (req, res) => {
  const conflict = await Booking.findOne({
    date: req.body.date, time: req.body.time, service: req.body.service, status: "Approved"
  });
  if (conflict) {
    return res.status(400).json({ error: "Slot already booked" });
  }
  const booking = new Booking({ ...req.body, status: "Pending" });
  await booking.save();
  res.json({ message: "Booking created", id: booking._id });
});
```

- **Officer Approval:** Officer dashboard lists pending bookings → approve/reject.
- **Citizen Notification:** Email sent with license/confirmation.

4.2.4 Online Payments & Finance

- **Frontend:** Citizen selects payment type → redirected to Stripe checkout.
- **Backend:**

```
app.post("/api/payments", async (req, res) => {
  const payment = new Payment({
    citizen_id: req.body.citizen_id,
    service: req.body.service,
    amount: req.body.amount,
    status: "Paid"
  });
  await payment.save();
  res.json({ message: "Payment successful", id: payment._id });
});
```

- **Integration:**
 - Stripe test environment used for secure payments.
 - System generates digital receipts (PDF).

4.2.5 HR & Payroll Module

- **Salary Calculation:**

```
function calculateSalary(base, overtimeHours, overtimeRate, deductions) {
  return base + (overtimeHours * overtimeRate) - deductions;
}
```
- **Payslip Generation:** Employees log in to download salary slips.

4.2.6 Inventory Management Module

- **Stock Control:**

```
app.post("/api/inventory/add", async (req, res) => {
  const item = new Inventory({
    item_name: req.body.item_name,
    quantity: req.body.quantity,
    reorder_level: req.body.reorder_level
  });
  await item.save();
  res.json({ message: "Item added", id: item._id });
});
```
- **Issue Items:** System deducts stock quantity and records issue details.
- **Reports:** Monthly usage and stock level reports generated.

4.3 API Structure

- **/api/complaints** – submit, view, update complaints.
- **/api/permits** – apply, approve, reject permits.
- **/api/bookings** – book service, check conflicts, approve/reject.
- **/api/payments** – handle payments, receipts.

- **/api/employees** – manage HR & payroll.
- **/api/inventory** – add, update, issue stock.

4.4 Authentication & Security

- **Role-based Access:** Admin, Citizen, Officer, HR, Finance.
- **Data Security:** HTTPS enforced for all requests.

4.5 Deployment

- **Database Hosting:** MongoDB Atlas cluster.
- **Version Control:** GitHub repository for collaborative development.
- **CI/CD:** Automatic build and deployment on GitHub push.

Chapter 5 – Testing

5.1 Test Case Design

5.1.1. Financial & Payment Module

Table 1- Test Case for Financial & Payment Module

ID	Description	Precondition	Test Steps	Expected Result
TC-FM-01	Verify property assessment entry	Officer logged in	1. Enter property details and assessment value. 2. Submit form.	Data saved in DB and linked to officer profile.
TC-FM-02	Update assessment record	Assessment exists	1. Edit assessment value. 2. Save changes.	Record updated with validation rules.
TC-FM-03	Auto calculate tax & fines	Property assessment exists	1. Add late payment. 2. View dashboard.	Correct tax with fine displayed.
TC-FM-04	Pay property tax online	Citizen logged in	1. Enter card details in payment gateway. 2. Submit payment.	Success message displayed, receipt generated.
TC-FM-05	Generate assessment reports	Officer logged in	1. Request PDF report.	Report generated with officer's sign and council details.
TC-FM-06	Shop registration workflow	Citizen logged in	1. Enter shop details. 2. Submit form.	Shop registration stored and linked to citizen profile.
TC-FM-07	Pay monthly shop rent	Shop registration approved	1. Enter payment details. 2. Submit payment.	Rent recorded and receipt displayed.

5.1.2. Property Assessment & Shop Module*Table 2- Test Case Property Assessment & Shop Module*

ID	Description	Precondition	Test Steps	Expected Result
TC-PA-01	Add property in multiple languages	Citizen logged in	1. Enter property details in Sinhala/English/Tamil. 2. Submit form.	Property saved in DB with multilingual support.
TC-PA-02	Citizen views property assessment	Property registered	1. Login as citizen. 2. Open dashboard.	Assessment details displayed.
TC-PA-03	Shop registration approval/rejection	Shop request exists	1. Officer reviews application. 2. Approve/Reject with remarks.	Status updated in citizen dashboard.
TC-PA-04	Citizen views shop registration status	Shop request submitted	1. Login as citizen. 2. Open shop registration dashboard.	Approved/Rejected status displayed.
TC-PA-05	Income/expense report generation	Officer logged in	1. Select report type. 2. Export to PDF.	Report generated successfully.

5.1.3 Service & License Permit System*Table 3- Test Case for Service & License Permit System*

ID	Description	Precondition	Test Steps	Expected Result
TC-LP-01	Apply for permit/license	Citizen logged in	1. Fill online form. 2. Submit application.	Permit request stored and linked to citizen profile.
TC-LP-02	Officer approves/rejects permit	Application exists	1. Officer reviews request. 2. Approve/Reject.	Status updated, citizen notified.
TC-LP-03	Citizen tracks permit status	Application submitted	1. Login as citizen. 2. View permit dashboard.	Real-time status displayed.

TC-LP-04	Generate permit reports	Admin logged in	1. Request report by category/date.	Permit report generated and exported.
TC-LP-05	Temporary PDF license download	Permit approved	1. Citizen selects approved permit. 2. Click “Download License”.	PDF license generated.

5.2 Test Results

- **Unit Testing:** All core functions (complaint submission, auto-tax, salary calculation) worked correctly.
- **Integration Testing:** Complaints flowed correctly from citizen submission → officer review → notification. Bookings correctly prevented duplicate slots.
- **System Testing:** Full workflows were validated; payments were processed, PDF licenses and receipts generated.
- **UAT Feedback:**
 - Citizens appreciated 24/7 online availability and email notifications.
 - Officers requested a filter option for complaint categories (added as improvement).
 - HR staff confirmed payroll accuracy but suggested adding a leave balance summary (to be implemented in future).

5.3 Bug Fixes and Improvements

During testing, several bugs were identified and fixed:

- **Issue:** PDF license not downloading properly.
 - **Fix:** Updated jsPDF library version and added async handling.
- **Issue:** Duplicate booking conflict not flagged when officer approved simultaneously.
 - **Fix:** Added server-side slot validation before approval.
- **Issue:** Employee payroll not updating after leave adjustment.
 - **Fix:** Corrected salary formula to re-calculate after leave record update.

Chapter 6 – Evaluation and Conclusion

6.1 Evaluation Against Objectives

Table 4- Evaluation Against Objectives

Objective	Evaluation Result
Provide a centralized web-based platform	Achieved – All modules integrated into one system with citizen, officer, HR, and admin dashboards.
Implement complaint management with tracking	Achieved – Citizens receive complaint IDs; officers update statuses; notifications sent.
Enable online permits and licenses	Achieved – Applications submitted digitally; approvals generate PDF licenses.
Provide secure online payments	Achieved – Payments processed via Stripe sandbox; receipts generated.
Automate HR and payroll	Achieved – Salaries calculated automatically; payslips generated for employees.
Enable multilingual access	Partially Achieved – Forms developed in Sinhala & English; Tamil interface still under development.
Integrate chatbot for bookings	Achieved – Chatbot booking tested successfully; conflict prevention works.
Provide notifications and reports	Achieved – Email notifications sent; finance and inventory reports generated.
Ensure scalability and security	Achieved – MERN stack

6.3 SWOT Analysis

Strengths

- Fully digitized complaints, permits, payroll, bookings, and payments.
- Cloud-hosted and accessible 24/7.
- Secure role-based access and encrypted data.
- Scalable design for adoption by other councils.
- Introduced chatbot booking – innovative for Sri Lankan councils.

Weaknesses

- Some modules (Tamil translations, advanced HR features) incomplete.

- Relies on stable internet; rural citizens may face difficulties.
- Officers require initial training to adapt to the system.

Opportunities

- Expansion to other local government bodies in Sri Lanka.
- Integration with Stripe platform for real payments.
- Adding mobile app for citizens.
- Extending modules (waste management, citizen surveys, inventory procurement).

Threats

- Resistance from staff accustomed to manual processes.
- Cybersecurity risks if not regularly updated.
- Dependence on cloud hosting and internet availability.